

Ambito de decision (Objetos / Arquitectura / Persistencia / Otro)	Componente/s impactado/s	Decisión	Otras Alternativas	Justificación de la decisión
Arquitectura	Pipeline CI/CD, GitHub Actions, Git Hooks, GitHub Projects, GitHub Pages, gestión de PRs	Implementar un pipeline unificado de CI/CD con validaciones tempranas, automatización de calidad, control de Git Flow y despliegue automático de ADRs	Procesos manuales de validación y despliegue	Centraliza controles de calidad y gobernanza del proyecto, asegura cumplimiento del flujo de trabajo, automatiza revisiones y documentación, y reduce errores de integración
Otro (Gestión del Proyecto)	Organización del equipo, asignación de tareas, reuniones y seguimiento del proyecto	Asignar roles y responsabilidades según fortalezas e intereses de los integrantes, con líderes y colaboradores por área	Organización sin roles fijos	Garantiza cobertura de todas las áreas del proyecto, define responsables explícitos, mejora la coordinación del equipo y aprovecha las capacidades e intereses de cada integrante
Objetos	Clases con datos sensibles de usuarios y personas	Implementar una interfaz Anonimizable para anonimizar información sensible ante solicitudes de supresión de datos	Eliminar entidades asociadas a la persona; separar datos sensibles en entidades independientes	Permite cumplir con requisitos de privacidad manteniendo la información histórica del sistema, con una implementación simple y de bajo impacto en el modelo actual
Objetos	Modelo de necesidades de entidades beneficiarias (Necesidad, NecesidadExtraordinaria, NecesidadRecurrente)	Modelar las necesidades mediante una clase abstracta Necesidad utilizando Template Method	No se documentan alternativas adicionales	Estandariza el comportamiento común de las necesidades y facilita la extensión del dominio incorporando nuevos tipos con lógica específica de cálculo
Objetos	ItemDonacion, gestión de stock y asignación de donaciones	Gestionar el stock mediante el atributo cantidadDisponible en ItemDonacion	Crear una entidad StockBien independiente	Permite mantener la trazabilidad entre los bienes asignados y sus donantes, facilitando el seguimiento del origen de cada donación
Objetos	Categoria, Subcategoria, Bien, Necesidad	Modelar categorías y subcategorías como entidades propias, utilizando Subcategoria como unidad mínima de asignación	Representar categorías y subcategorías mediante atributos simples (String o Enum)	Mantiene un modelo de dominio claro y reutilizable, centraliza reglas comunes en Categoria y permite realizar asignaciones y necesidades a nivel de Subcategoria
Objetos	Direccion, Localidad, Provincia, Pais	Modelar la ubicación geográfica mediante una composición jerárquica Direccion, Localidad, Provincia, Pais	Incluir localidad, provincia y país como atributos simples dentro de Direccion	Separa responsabilidades, representa mejor la estructura geográfica del dominio y facilita la reutilización y extensión del modelo
Objetos	Persona, Humana, Juridica, Genero	Modelar una clase abstracta Persona con especializaciones Humana y Juridica	Modelar personas humanas y jurídicas como clases independientes sin abstracción común	Centraliza los atributos compartidos, separa claramente las características específicas de cada tipo de persona y permite tratarlas de forma uniforme cuando sea necesario
Objetos	EventoNotificable, eventos de dominio y generación de notificaciones	Modelar los eventos notificables mediante una clase abstracta EventoNotificable que genere objetos Notificacion	Utilizar una interfaz con un método generarMensajes()	Permite generar notificaciones completas asociando destinatarios y mensajes, reutilizar comportamiento común entre eventos y facilitar la incorporación de nuevos tipos de notificaciones
Objetos	EventoDeDonacion, eventos de asignación y recepción de donaciones, generación de notificaciones	Modelar los eventos de donación mediante una clase abstracta EventoDeDonacion aplicando Template Method	Separar cada evento en EventoDonante y EventoBeneficiario independientes	Permite representar un único evento con múltiples destinatarios, reutilizar lógica común y facilitar la extensión del modelo para nuevas notificaciones relacionadas con donaciones
Arquitectura	Comunicación entre microservicios y Servicio de Notificaciones	Implementar comunicación asíncrona basada en eventos y colas de mensajería	Comunicación síncrona mediante APIs	Reduce el acoplamiento entre servicios, mejora la escalabilidad y tolerancia a fallos, y permite procesar eventos sin bloquear los servicios productores
Objetos	Notificacion, gestión de estados y reintentos de envío	Modelar los estados de las notificaciones mediante un enum EstadoNotificacion (PENDIENTE, ENVIADA, FALLIDA)	Implementar el patrón State con una jerarquía de clases EstadoNotificacion	Proporciona una solución simple y suficiente para gestionar el ciclo de vida de las notificaciones y los fallos básicos de envío, con menor complejidad de implementación
Objetos	MedioDeContacto, Correo, Telefono, Whatsapp, envío de notificaciones	Modelar los medios de contacto mediante una clase abstracta MedioDeContacto y especializaciones concretas	Representar los medios mediante un enum MedioDeContacto	Permite un diseño polimórfico y extensible, encapsula la lógica específica de cada medio de comunicación y desacopla el envío de mensajes mediante una interfaz especializada
Arquitectura	Notificacion, MedioDeContacto, NotificacionSender, NotificacionRouter y adaptadores de envío	Desacoplar el envío de notificaciones mediante Double Dispatch y un Router/Facade en infraestructura	Resolver el envío con condicionales (instanceof) en servicios de aplicación	Mantiene el dominio independiente de APIs externas, evita lógica condicional dependiente de tipos concretos y facilita la extensibilidad y el testing de la lógica de negocio
Otro (Testing)	Pruebas unitarias de notificaciones, Notificacion, NotificacionSender y adaptadores	Utilizar Mockito para crear mocks y simular dependencias externas en las pruebas unitarias	Implementar clases Stub/Fake manualmente	Permite probar la lógica de negocio de forma aislada, rápida y determinística, reduciendo código de prueba y facilitando la simulación de escenarios de éxito y fallo